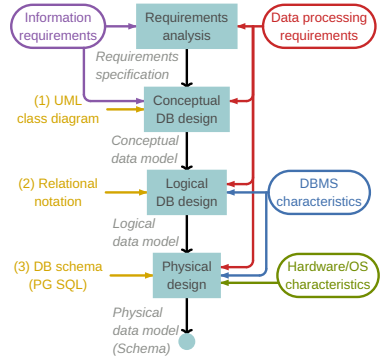
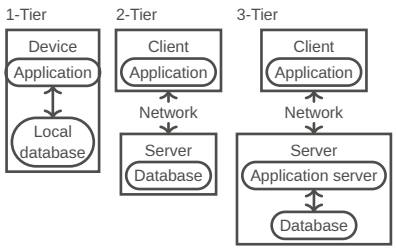


GLOSSARY

German term	English term	Definition
Impedance-Mismatch	Impedance-Mismatch	Mismatch between data structures at the application and database levels
System-/ Datenkatalog	System/data catalog	Contains metadata about database objects, e.g. tables and schemas.
Daten-bankschema	Database schema	Structure of a database describing the organization of data and relationships.
Datenbasis	Data store	The physical storage location
Surrogate Key	Surrogate Key	Artificially generated PK
Referentielle Integrität	Referential integrity	A foreign key must reference a value in the referenced table or NULL
Datenunab-hängigkeit	Data indepen-dence	Ability to change data in a DB without having to change applications
Data Pages	Data Pages	Smallest data storage units of a DB
Heaps	Heaps	Unsorted data organization
Semantische Integrität	Semantic integrity	Data is correct in both syntax and content, especially after transactions
Data dictionary	Data dictionary	Central collection of metadata about the data in the DBMS

DATABASE MODELS

German term	English term	Definition
Hierarchisch	Hierarchical	Data is arranged in a tree-like structure
Netzwerk	Network	More flexible structure than hierarchical; allows multiple paths between entities
Objektorientiert	Object-oriented	Stores data and its behavior as objects
Objektrelational	Object-relational	Combines object-oriented + relational principles
Relational	Relational	Stores data in tables (relations) and manages relationships using keys



DATABASE SYSTEM (DBS)

Consists of a DBMS and data stores

DATABASE MANAGEMENT SYSTEM (DBMS)

- (A) Transactions
- (C) Consistency
- (I) Multi-user operation
- (D) Large amounts of data
- (D) Security
- Data types
- Query language
- Backup & Recovery
- Freedom from redundancy
- Encapsulation

ANSI MODEL

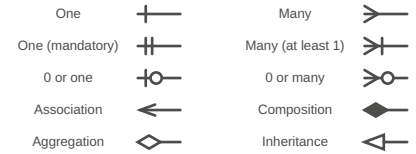
Logical level: Logical structure of the data
Internal level: Storage structures, defined by the internal schema (relationships, tables, etc.)
External level: A user class's view of a subset of the DB, defined by an external schema
Mapping: A more or less complex mapping is required between the levels

RELATIONAL MODEL

PKs are underlined, FKs are *italicized*

```
tabellenname (
  id SERIAL PRIMARY KEY,
  grade DECIMAL(2,1) NOT NULL,
  fk INT FOREIGN KEY REFERENCES t2,
  u VARCHAR(9) DEFAULT CURRENT_USER,
);
```

UNIFIED MODELING LANGUAGE (UML)



Complete: All subclasses are defined
Incomplete: Additional subclasses are allowed
Disjoint: An instance of exactly one subclass
Overlapping: Can be an instance of multiple overlapping subclasses

NORMALIZATION

1NF: Atomic attribute values: split ***track***

id	track
1	Fugazi: Song #1

⇒

id	interpret	titel
1	Fugazi	Song #1

2NF: Non-key attributes fully depend on the key.

If the PK is atomic, 2NF is satisfied. In the example, not all PK attributes are needed to uniquely identify ***album***

track	cd_id	album	titel
1	1	Repeater	Turnover
2	1	Repeater	Song #1

⇒ track

track	cd_id	titel
1	1	Turnover
2	1	Song #1

cd

id	album
1	Repeater

3NF: No transitive dependencies: ***land*** depends on ***interpret***

id	album	interpret	land
1	Repeater	Fugazi	USA
2	Red Medicine	Fugazi	USA

⇒ cd

id	album	interpret
1	Repeater	1

interpret

id	name	land
1	Fugazi	USA

BCNF: Only dependencies on the key
(Full) functional dependency: B depends on A; for each value of A there is exactly one value of B ($A \rightarrow B$)
Partial functional dependency: B depends on A, but also on part of a composite key.
Transitive dependency: B depends on attribute A, C depends on B ($A \rightarrow B \wedge B \rightarrow C \Rightarrow A \rightarrow C$)

Denormalization: Reverting to a lower NF (improves performance and reduces join complexity)

ANOMALIES

Insertion anomaly, deletion anomaly, update anomaly

DATA CONTROL LANGUAGE (DCL)

```
<role> ::= 'ALTER ROLE' <rname> <priv> { ',' <priv> }
<grant> ::= 'GRANT' <actions> 'ON' <object> 'TO' <grantees>
[ 'WITH GRANT OPTION' ]
<revoke> ::= 'REVOKE' [ 'GRANT OPTION FOR' ] <actions> 'ON'
<object> 'FROM' <grantees> ( 'CASCADE' | 'RESTRICT' )
<priv> ::= [ 'NO' ] ( 'CREATEDB' | 'CREATEROLE' | 'INHERIT' )
<action> ::= 'ALL' | 'SELECT' | 'DELETE' | 'TRIGGER' |
(( 'INSERT' | 'UPDATE' | 'REFERENCES' ) '(' <columns> ')' )
<actions> ::= <action> { ',' <action> }
<grantees> ::= <role_name> { ',' <role_name> }
<object> ::= 'TABLE' | 'COLUMN' | 'VIEW' | 'SEQUENCE' | 'DATABASE'
| 'FUNCTION' | 'SCHEMA'
```

With `WITH GRANT OPTION`: The grantee can grant access to other users. → `REVOKE ... CASCADE;`

```
CREATE ROLE u WITH LOGIN PASSWORD ''; -- user
GRANT INSERT ON TABLE t TO u WITH GRANT OPTION;
ALTER ROLE u CREATEROLE, CREATEDB, INHERIT;
CREATE ROLE r; -- group
GRANT r TO u; -- put user u in group r
REVOKE CREATE ON SCHEMA s FROM r;
CREATE ROLE u PASSWORD '' IN ROLE r; -- equivalent
```

READ-ONLY USER

```
-- creating
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
CREATE ROLE u WITH LOGIN ENCRYPTED PASSWORD ''
  NOINHERIT; -- don't inherit privileges
GRANT SELECT ON ALL TABLES IN SCHEMA public TO u;
-- read all new tables (also created by others):
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT
  SELECT ON TABLES TO u;
-- deleting
REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM u;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  REVOKE SELECT ON TABLES FROM u;
DROP USER u;
```

ROW-LEVEL SECURITY (RLS)

```
CREATE TABLE exams (
  id SERIAL, -- other fields...
  teacher VARCHAR(60) DEFAULT current_user
);
CREATE POLICY teachers_see_own_exams ON exams
  FOR ALL TO PUBLIC USING (teacher = current_user);
ALTER TABLE exams ENABLE ROW LEVEL SECURITY;
```

DATA DEFINITION LANGUAGE (DDL)

Important: Do not forget NOT NULL where required

```
CREATE SCHEMA s;
CREATE TABLE t (
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE,
  grade DECIMAL(2,1) NOT NULL,
  fk INT FOREIGN KEY REFERENCES t2.id
    ON DELETE CASCADE,
  added TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  u VARCHAR(9) DEFAULT CURRENT_USER,
  CHECK (grade between 1 and 6)
);
ALTER TABLE t2 ADD CONSTRAINT c PRIMARY KEY (a, b);
TRUNCATE/DROP TABLE t;
```

INHERITANCE

One table per subclass and superclass:

```
CREATE TABLE sup ( -- 3.a
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE
);
CREATE TABLE sub1 (
  id SERIAL PRIMARY KEY,
  age INT
);
CREATE TABLE sub2 (
  id SERIAL PRIMARY KEY
);
ALTER TABLE sub1 ADD CONSTRAINT id FOREIGN KEY
  REFERENCES sup (id);
ALTER TABLE sub2 ADD CONSTRAINT id FOREIGN KEY
  REFERENCES sup (id);
```

One table per subclass: Contains the respective subclass attributes

```
CREATE TABLE sub1 ( -- 3.b
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE,
  age INT
);
CREATE TABLE sub2 (
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE
);
```

Single table for the superclass: Contains all attributes

```
CREATE TABLE sup ( -- 3.c
  id SERIAL PRIMARY KEY,
  name TEXT UNIQUE,
  age INT
);
```

JUNCTION TABLES

```
CREATE TABLE a_b(
  a INTEGER REFERENCES a(id),
  b INTEGER REFERENCES b(id),
  PRIMARY KEY(a, b)
);
```

VIEWS

Results are queried dynamically each time

```
CREATE VIEW v (id, u) AS SELECT id, u FROM t;
-- complex query
CREATE VIEW cheap_restaurant_view AS
WITH big_restaurant AS (
  SELECT * FROM restaurant
  WHERE anzahl_plaetze ⩾ 20
)
SELECT r.name AS restaurant_name, s.name,
  MIN(g.preis) AS cheap_gericht
FROM big_restaurant r
LEFT JOIN skigebiet s ON (s.id = r.skigebiet_id)
LEFT JOIN menukarte m ON (r.id = m.restaurant_id)
LEFT JOIN menu_gericht mg ON (m.id = mg.menu_id)
LEFT JOIN gericht g ON (g.id = mg.gericht_id)
WHERE ist_tagesmenu = true
GROUP BY r.id, s.id, restaurant_name
HAVING MIN(g.preis) ⩾ 3
ORDER BY cheap_gericht;
```

UPDATABLE VIEW

Views are updatable when these criteria are met:
- A single "base table"
- No aggregates, DISTINCT, GROUP BY, or HAVING
- All columns must map directly to the original table

MATERIALIZED VIEW

Stores the result on disk

```
CREATE MATERIALIZED VIEW mv AS SELECT * FROM t;
REFRESH MATERIALIZED VIEW mv; -- refresh results
```

TEMPORARY TABLES

```
CREATE TEMPORARY TABLE temp_products (
  id SERIAL PRIMARY KEY,
  product_name TEXT
);

INSERT INTO temp_products (product_name) VALUES
  ('Product A'), ('Product B'), ('Product C');

SELECT ts.product_name, ts.quantity FROM
  temp_sales ts JOIN temp_products tp ON
  ts.product_name = tp.product_name;
```

DATA TYPES

```
CREATE TYPE grade AS ENUM('A','B','C','D','E','F');
NUMERIC(4, 2) /* 99.99 */ NUMERIC(2, 1) /* 9.9 */
VARCHAR(5) /* 'abcde' */  CHAR(5) /* 'abcde' */
```

THREE-VALUED LOGIC (CURSED)

```
SELECT NULL IS NULL; -- true
SELECT NULL = NULL;  -- [unknown]
```

German term	English term	Definition
INTEGER/INT	INTEGER/INT	Integer (4 bytes)
BIGINT	BIGINT	Large integer (8 bytes)
SMALLINT	SMALLINT	Small integer (2 bytes)
REAL	REAL	Single precision float (4 bytes)
NUMERIC(pre-cision, scale)	NUMERIC(pre-cision, scale)	Exact numeric of selectable precision Alias for `DECIMAL(precision, scale)`
DOUBLE PRE-CISION	DOUBLE PRE-CISION	Double precision float (8 bytes)
SERIAL	SERIAL	Auto-incrementing integer (4 bytes)
BIGSERIAL	BIGSERIAL	Auto-incrementing large integer (8 bytes)
SMALLSERIAL	SMALLSERIAL	Auto-incrementing small integer (2 bytes)
CHARACTER/ CHAR(size)	CHARACTER/ CHAR(size)	Fixed-length, blank-padded string
VAR-CHAR(size)	VAR-CHAR(size)	Variable-length, non-blank-padded string
TEXT	TEXT	Variable-length character string
BOOLEAN	BOOLEAN	Logical Boolean (true/false)
DATE	DATE	Calendar date (year, month, day)
TIME	TIME	Time of day (no time zone)
TIMESTAMP	TIMESTAMP	Date and time (no time zone)
TIMESTAMP WITH TIME ZONE	TIMESTAMP WITH TIME ZONE	Date and time with time zone
INTERVAL	INTERVAL	Time interval
JSON	JSON	JSON data
UUID	UUID	Universally unique identifier
ARRAY OF base_type	ARRAY OF base_type	Array of values

CASTING

Explicit

```
CAST(5 AS float8) = 5::float8
SELECT 'ABCDEFG'::NUMERIC; -- error
SELECT SAFE_CAST('ABCDEFG' AS NUMERIC); -- NULL
```

Implicit

```
SELECT 5 + 3.2; -- 5 is cast to 5.0 (numeric)
SELECT 'Number ' || 42; -- 42 is cast to '42'
SELECT true AND 1; -- 1 is treated as true
SELECT CURRENT_TIMESTAMP + INTERVAL '1 day';
-- CURRENT_TIMESTAMP to date ^
SELECT '100'::text + 1; -- '100' is cast to 100
```

DATA MANIPULATION LANGUAGE (DML)

```
<select> ::= [ 'WITH' [ 'RECURSIVE' ] <with_query> [',' ...] ]
'SELECT' [ 'ALL' | 'DISTINCT' [ 'ON' ( <expression>
[',' ...] ) ] ]
    [ { '*' | <expression> [ [ 'AS' ] <output_name> ] }
[',' ...] ]
    [ 'FROM' <from_item> [',' ...] ]
    [ 'WHERE' <condition> ]
    [ 'GROUP BY' [ 'ALL' | 'DISTINCT' ] <grouping_elem>
[',' ...] ]
    [ 'HAVING' <condition> ]
    [ 'WINDOW' <window_name> 'AS' ( <window_def> ) [',' ...] ]
    [ { 'UNION' | 'INTERSECT' | 'EXCEPT' } [ 'ALL' | 'DISTINCT' ]
<select> ]
    [ 'ORDER BY' <expression> [ 'ASC' | 'DESC' | 'USING' <op> ]
[ 'NULLS' { 'FIRST' | 'LAST' } ] [',' ...] ]
    [ 'LIMIT' { <count> | 'ALL' } ]
    [ 'OFFSET' <start> [ 'ROW' | 'ROWS' ] ]

<from_item> ::= <table> [ '*' ] [ [ 'AS' ] <alias> [ ( <col_alias>
[',' ...] ) ] ]
    [ 'LATERAL' ] ( <select> ) [ [ 'AS' ] <alias> [ ( <col_alias>
[',' ...] ) ] ]
    <with_query_name> [ [ 'AS' ] <alias> [ ( <col_alias>
[',' ...] ) ] ]
    <from_item> <join_type> <from_item> { 'ON' <join_condition> |
'USING' ( <join_column> [',' ...] ) [ 'AS' <join_using_alias> ] }
    <from_item> 'NATURAL' <join_type> <from_item>
    <from_item> 'CROSS JOIN' <from_item>

<with_query> ::= <name> [ ( <col_name> [',' ...] ) ] 'AS'
( <select> | <values> | <insert> | <update> | <delete> | <merge> )
        [ 'USING' <cycle_path_col_name> ]
```

JOINS

users (*u*)

id	name
1	Alice
2	Bob

actions (*a*)

id	uid	action
7	1	LOGIN
8	2	VIEW
9	4	LOGIN

INFO: FK ***uid*** omitted from the query results below to save space

INNER JOIN

Rows that match in both tables

```
SELECT u.*, a.* FROM u INNER
JOIN a ON u.id = a.uid;
```

1	1	Alice	7	LOGIN

EQUI JOIN

Like Inner Join

```
SELECT u.*, a.* FROM u JOIN a
ON u.id = a.uid;
```

1	1	Alice	7	LOGIN

NATURAL JOIN

Like Inner Join, but without duplicates

```
SELECT u.*, a.* FROM u NATURAL
JOIN a ON u.id=a.uid;
```

1	1	Alice	7	LOGIN

SEMI JOIN

Only rows from a for which b must match

```
SELECT * FROM u WHERE EXISTS
(SELECT 1 FROM a WHERE u.id = a.uid);
```

1	1	Alice

ANTI JOIN

Only rows from a for which b must not match

```
SELECT * FROM u WHERE NOT EXISTS
  (SELECT 1 FROM a WHERE u.id = a.uid);
```

1	2	Bob

LEFT OUTER JOIN

All rows from both tables; NULL for b if there is no match

```
SELECT u.*, a.* FROM u LEFT
JOIN a ON u.id = a.uid;
```

1	1	Alice	7	LOGIN
2	2	Bob		

RIGHT OUTER JOIN

All rows from both tables; NULL for a if there is no match

```
SELECT u.*, a.* FROM u RIGHT
JOIN a ON u.id = a.uid;
```

1	1	Alice	7	LOGIN
3			8	VIEW
4			9	LOGIN

FULL OUTER JOIN

All rows from both tables; NULL if there is no match

```
SELECT u.*, a.* FROM u FULL
OUTER JOIN a ON u.id = a.uid;
```

1	1	Alice	7	LOGIN
2	2	Bob		
3			8	VIEW
4			9	LOGIN

CROSS JOIN

Returns all possible combinations of two tables.

```
SELECT * FROM u CROSS JOIN a;
```

1	1	Alice	7	LOGIN
2	1	Alice	8	VIEW
3	1	Alice	9	LOGIN
4	2	Bob	7	LOGIN
5	2	Bob	8	VIEW
6	2	Bob	9	LOGIN

UNION

"Combines" two SELECTs without duplicates. Prerequisite: Columns must contain similar data types. **Union all** is like Union, but with duplicates ⇒ Recursive CTEs

```
SELECT name FROM u UNION SELECT action FROM a;
```

1	Alice
2	Bob
3	LOGIN
4	VIEW

LATERAL JOIN

Allows subqueries referencing the other tables

```
SELECT u.*, x.action FROM u JOIN LATERAL
  (SELECT * FROM a WHERE a.uid =≠ u.id)
  AS x ON TRUE;
```

1	LOGIN

INSERT

```
INSERT INTO t (added, grade)
  VALUES ('2002-10-10', 1) RETURNING id;
```

UPDATE

```
UPDATE t SET grade = grade+1, name='' WHERE id = 1;
```

SUBQUERIES

```
SELECT * FROM t WHERE grade > ANY
  (SELECT g FROM t2);
SELECT * FROM t WHERE EXISTS (SELECT g FROM t2);
-- ALL, ANY, IN, EXISTS, =
```

GROUP BY

```
SELECT id, COUNT(*) FROM t
  GROUP BY grade, id HAVING COUNT(*) > 2;
```

WHERE

```
BETWEEN 1 AND 5; LIKE '___%'; AND; IS (NOT) NULL
IN (1, 5)      ; LIKE '%asd'; OR ;
```

AGGREGATE FUNCTIONS

```
COUNT   ; SUM   ; MIN   ; MAX   ; AVG
```

OTHER FUNCTIONS

```
COALESCE(a1, a2, ...); -- returns first non-null arg
```

COMMON TABLE EXPRESSIONS (CTE)

– Allow row-by-row output
– Allow queries to act as parameters
– Can be recursive

```
-- normal
WITH cte AS (SELECT * FROM t) SELECT * FROM cte;
WITH tmp(id, name) AS (SELECT id, name FROM t)
  SELECT id, name FROM tmptable;
-- rekursiv
WITH RECURSIVE q AS (
    SELECT * FROM t WHERE grade > 1
  UNION ALL SELECT * FROM t INNER JOIN q ON
    q.u = t.name
) SELECT id as 'ID' FROM q;
```

WINDOW FUNCTIONS

```
SELECT id, RANK() OVER
  (ORDER BY grade DESC) as r FROM t;
SELECT id, u, LAG(name, 1) OVER
  (PARTITION BY fk ORDER BY id DESC) FROM t;
-- PERCENT/DENSE_RANK(), FIRST_VALUE(v),
LAST_VALUE(n)
-- NTH_VALUE(v,n), NTILE(n), LEAD(v,o), ROW_NUMBER()
```

RELATIONAL ALGEBRA

$\pi_{R1,R4}(R)$ `SELECT R1,R4 FROM R;` (Projection)
$\sigma_{R1>30}(R)$ `SELECT * FROM R WHERE R1 > 30;` (Selection)
$\rho_{a \leftarrow R}$ `SELECT * FROM R AS a;` (Renaming/Alias)
$R \times S$ `SELECT * FROM R,S;` (Cartesian product)
$R \bowtie_{A=B} S$ `SELECT * FROM R JOIN S ON R.A=S.B;` (Join)

INDEX

	B-Tree	*Hash*	*BRIN*	*ISAM*
Equality queries	✓	✓	×	✓
Range Queries	✓	×	✓	×
Sorted data	✓	×	✓	✓
Large tables	⋆	for =	✓	✓
Frequent queries	✓	⋆	✓	×
Direct access via PK	✓	✓	×	⋆
Overflow pages	✓	✓	×	✓

```
CREATE INDEX i ON t/*USING BTREE*/ (grade,UPPER(u));
CREATE INDEX j ON t (fk) INCLUDE (added) WHERE fk>4;
DROP INDEX i;
```

TRANSACTIONS

Note: PostgreSQL has no nested transactions.
Atomicity: All or nothing
Consistency: A consistent state is preserved
Isolation: A transaction is isolated from other transactions
Durability: Changes are persistent

```
BEGIN;  SAVEPOINT s;
COMMIT; ROLLBACK /*TO SAVEPOINT s*/;
```

ISOLATION

```
SET TRANSACTION ISOLATION LEVEL ...; -- transaction
SET SESSION CHARACTERISTICS AS TRANSACTION
  ISOLATION LEVEL ...; --  session
```

READ UNCOMMITTED: Reads are not synchronized (no read lock); reads ignore all locks

READ COMMITTED: Reads are synchronized briefly/temporarily (default); write locks last for the whole transaction, read locks only briefly
REPEATABLE READ: Individual row accesses are synchronized; read and write locks last for the whole transaction
SERIALIZABLE: Full isolation according to ACID

	Read Uncommitted	*Read Committed*	*Repeatable Read*	*Serializable*
Dirty Write	⋆	⋆	⋆	×
Dirty Read	✓	×	×	×
Lost Update	✓	✓	×	×
Fuzzy Read	✓	✓	×	×
Phantom Read	✓	✓	✓	×
Read Skew	✓	✓	×	×
Write Skew	✓	✓	✓	⋆
Durability	✓	✓	×	×
Atomicity	×	×	✓	✓

* Only possible in SQL92; PSQL >= 9.1 prevents this
Dirty Read: Reading data from uncommitted transactions
Fuzzy Read: Different values when reading the same data repeatedly (modified by another transaction)
Phantom Read: Rows inserted/deleted by another transaction
Read Skew: Reading data that changes during the transaction
Write Skew: Multiple transactions read data and modify it
Deadlock: Multiple transactions block each other while waiting for the same resource
Cascading Rollback: A transaction fails and all dependent transactions must also be rolled back

	Serializable	*Deadlocks*	*Cascading RollB.*	*Conflict rollback*	*High concurrency*	*Realistic*
Two-Phase Locking	✓	✓	✓	×	×	×
Strict 2PL	✓	✓	×	×	×	✓
Pre-claiming 2PL	✓	×	×	×	×	×
Validation-based	✓	×	✓	✓	✓	✓
Time-stamp-based	✓	×	✓	✓	✓	✓
Snapshot Isolation	×	⋆	×	✓	✓	✓
SSI	✓	⋆	×	✓	✓	✓

* Deadlock in PSQL with Snapshot Isolation

SQL example

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE accounts SET balance = balance - 100.00
    WHERE name = 'Alice';
SAVEPOINT my_savepoint;
UPDATE accounts SET balance = balance + 100.00
    WHERE name = 'Bob';
ROLLBACK TO my_savepoint;
UPDATE accounts SET balance = balance + 100.00
    WHERE name = 'Wally';
COMMIT;
```

TWO-PHASE LOCKING (2PL)

Ensures transaction isolation
1) Growing Phase: The transaction can acquire new locks but cannot release any
2) Shrinking Phase: Locks can be released, but no new locks can be acquired

Strict 2PL: Transactions release locks only after commit
Preclaiming 2PL: All locks are acquired at the start of the transaction

S**hared **Lock: Read access (multiple transactions)
EX**clusive **Lock: Write & read access (one transaction)
Starvation: A transaction never gets the chance to finish its work because locks keep blocking it

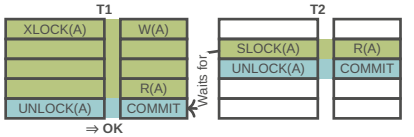
OPTIMISTIC LOCKING

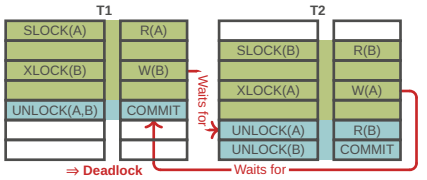
Transactions operate without initial locks. Check for conflicts at the end → roll back changes.

PESSIMISTIC LOCKING (PRECLAIMING 2PL)

Transactions request locks immediately so that other transactions cannot access or modify the same data simultaneously.

Growing phase **Shrinking phase**





SERIALIZABILITY

Serial schedule: Executes transactions without interleaving
Not serializable:

S1=R1(x)R2(x)W1(x)R1(y)W2(x)W1(y)



Conflicting pairs:

R1(x) < W2(x) R2(x) < W1(x)

Conflict-serializable:
r1(b)r2(b)w2(b)r2(c)r2(d)w3(a)r4(d)r3(b)w4(d)r5(c)r5(a)w4(c)
Conflict-equivalent serial schedule:
r1(b)r2(b)w2(b)r2(c)r2(d)w3(a)r3(b)r5(c)r5(a)r4(d)w4(d)w4(c)



German term	*English term*	*Definition*
Seriell	Serial	All transactions in a schedule are ordered
Konfliktäquivalent	Conflict-equivalent	The order of all pairs of conflicting actions is the same in both schedules
Konfliktserialisierbar	Conflict-serializable	A schedule is conflict-equivalent to a serial schedule

FULL BACKUP

Exact copy of the entire DB

INCREMENTAL BACKUP

Backs up only data changed since the last backup.

LOGICAL BACKUP (SQL DUMP)

Does not block transactions. For medium-sized datasets; compatible with new PG versions and other machines.

PHYSICAL BACKUP (FILE SYSTEM)

The database must be stopped; faster than a logical backup; works only with the same PG "major version".

MULTI-VERSION CONCURRENCY CONTROL (MVCC)

Allows multiple transactions to run concurrently. Each change creates a new version of the data. Readers see older versions, while writers see the latest versions.

WRITE-AHEAD LOG (WAL)

Writes transaction changes to the log, then logs the commit, then updates the DB. Can be replayed after a crash
LSN, TaID, PageID, Redo, Undo, PrevLSN

SQL EXAMPLES

```
CREATE TABLE pferd (
  pnr SERIAL PRIMARY KEY,
  name TEXT,
  alter INT,
  zuechternr INT REFERENCES stall.pk,
  vaternr INT REFERENCES pferd.pk
);
CREATE TABLE stall (
  zuechternr SERIAL PRIMARY KEY,
  name TEXT,
  plz INT,
  ort TEXT,
  strasse TEXT
);

-- Welche Züchter haben in ihren Ställen mindestens 1
Kind von dem Vater mit Namen "Hermes"

-- Eleganteste anfrage unkorreliert
SELECT s.name FROM staelle s
WHERE s.zuechternr IN (
  SELECT p.zuechternr
  FROM pferde p
  JOIN pferde p2 ON p2.pnr = p.vaternr
  WHERE p2.name = 'Hermes'
);
-- Kürzeste anfrage
SELECT DISTINCT s.name FROM staelle s
JOIN pferde p ON p.zuechternr = s.zuechternr
JOIN pferde p2 ON p2.pnr = p.vaternr
WHERE p2.name = 'Hermes';
--
SELECT DISTINCT s.name FROM staelle s
JOIN pferde p ON p.zuechternr = s.zuechternr
WHERE EXISTS (
  SELECT vaternr FROM pferde p2
  WHERE p2.pnr = p.vaternr AND p2.name = 'Hermes'
);

-- RECURSIVE CTE
WITH RECURSIVE tens AS (
   SELECT 1 as n
 UNION ALL
   SELECT n+1 FROM tens
) SELECT n FROM tens limit 10;
-- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

B-TREE

